VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Srushti Sunkad (1BM23CS341)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Sep-2024 to Jan-2025

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Srushti Sunkad (1BM23CS341),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Megha J Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	30-01-2026	Write a python program to import and export data using Pandas library functions	4-7
2	06-02-2026	Demonstrate various data pre-processing techniques for a given dataset	8-12
3	13-02-2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	12-14
4	27-02-2026	Build Logistic Regression Model for a given dataset	15-17
5	06-03-2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	17-19
6	13-03-2026	Build KNN Classification model for a given dataset.	19-22
7	08-04-2026	Build Support vector machine model for a given dataset	23-25
8	08-04-2026	Implement Random forest ensemble method on a given dataset.	25-27
9	08-04-2026	Implement Boosting ensemble method on a given dataset.	28-29
10	10-04-2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	30-32
11	10-04-2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	32-34

Github Link: GitHub - SrushtiSunkad630/MACHINE-LEARNING-LAB · GitHub

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot

30/01/26 Lab-1 ①

Q1). Demonstrate various data pre-processing techniques for a given data.

Write a python code, consider filename as "housing.csv"

1). To load.csv file into the data frame

=> df = ~~pd.csv~~ pd.read_csv("housing.csv")

2) To display information of all columns

=> print ("Column Information:")
print (df.describe")
df.info()

3) To display statistical information of all numerical

=> print (df.describe())

4) To display the count of unique labels for "Ocean Proximity" column.

=> print(df["Ocean Proximity"].value_counts())

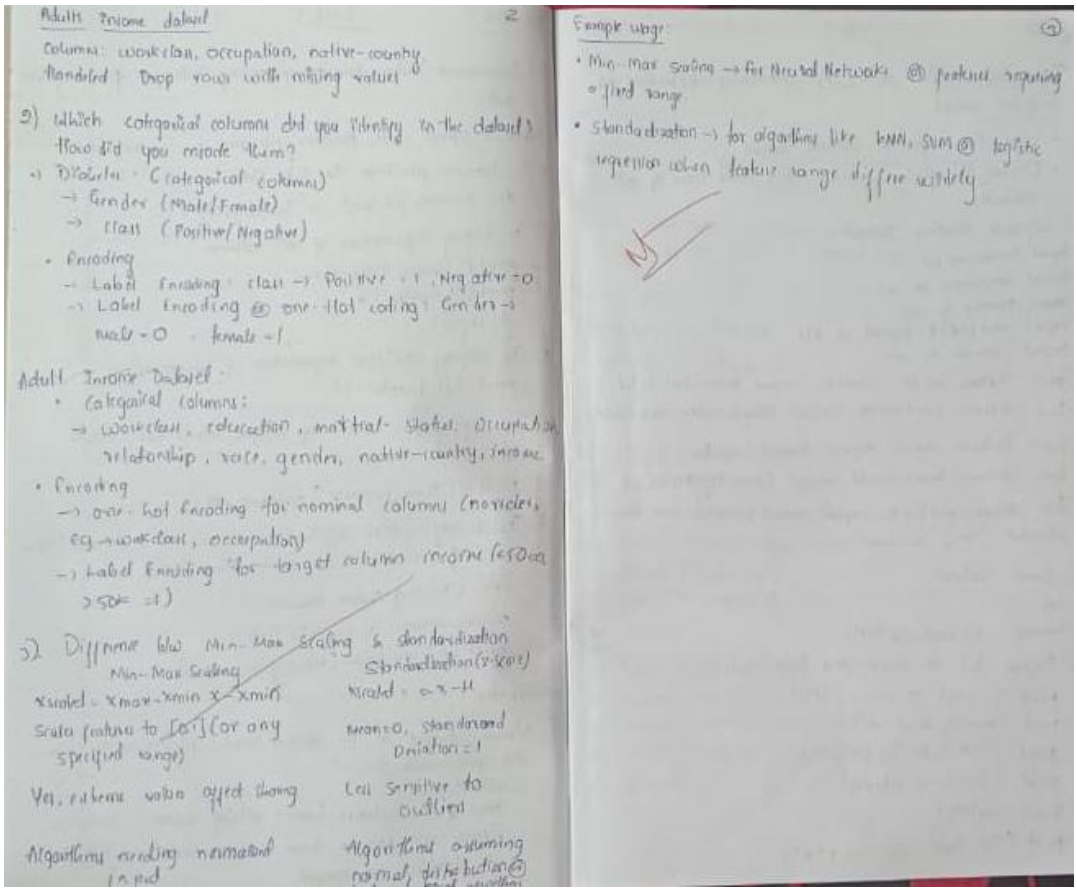
5) To display which attributes (columns) in a dataset have missing values count greater than zero.

=> print ("missing Values Columns:")
missing_values = df.isnull().sum()
print (missing_values[missing_values>0])

Diabetes

① Which columns in the dataset had missing values? How did you handle them?

=> ① Diabetes
None of the columns having missing values in diabetes. Since all columns have complete data, no imputation or deletion was needed.



Adults Income dataset

Columns: workclass, occupation, native-country

Handled: Drop rows with missing values

2) Which categorical columns did you identify in the dataset? How did you encode them?

→ Dataset - (categorical columns)

→ Gender (Male/Female)

→ Class (Positive/Negative)

- Encoding

→ Label Encoding: class → Positive = 1, Negative = 0

→ Label Encoding or one-Hot coding: Gender → male = 0, female = 1

Adult Income Dataset

- Categorical columns:

→ workclass, education, marital-status, occupation, relationship, race, gender, native-country, income

- Encoding

→ one-hot Encoding for nominal columns (no order)

eg → workclass, occupation)

→ Label Encoding for target column income (<50K = 0, >50K = 1)

3) Difference b/w Min-Max Scaling & standardization

Min-Max Scaling	Standardization (z-score)
$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$	$x_{scaled} = \frac{x - \mu}{\sigma}$
Scale feature to [0,1] (or any specified range)	mean = 0, standard Deviation = 1
Yes, extreme values affect scaling	Less sensitive to outliers
Algorithms needing normalized input	Algorithms assuming normal distribution

Example usage:

- Min-Max Scaling → for Neural Networks @ features requiring a fixed range.
- Standardization → for algorithms like KNN, SVM @ logistic regression when feature range differ widely

Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

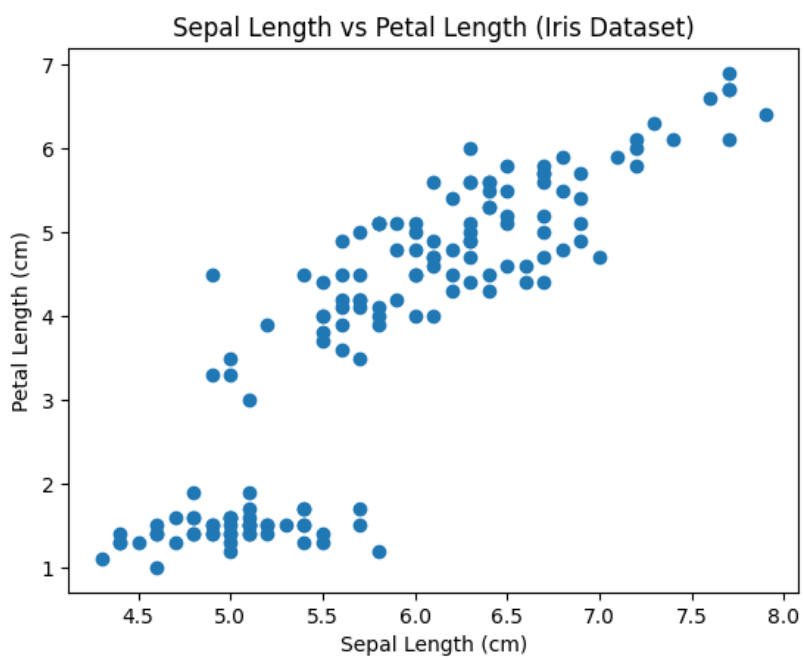
```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```



```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

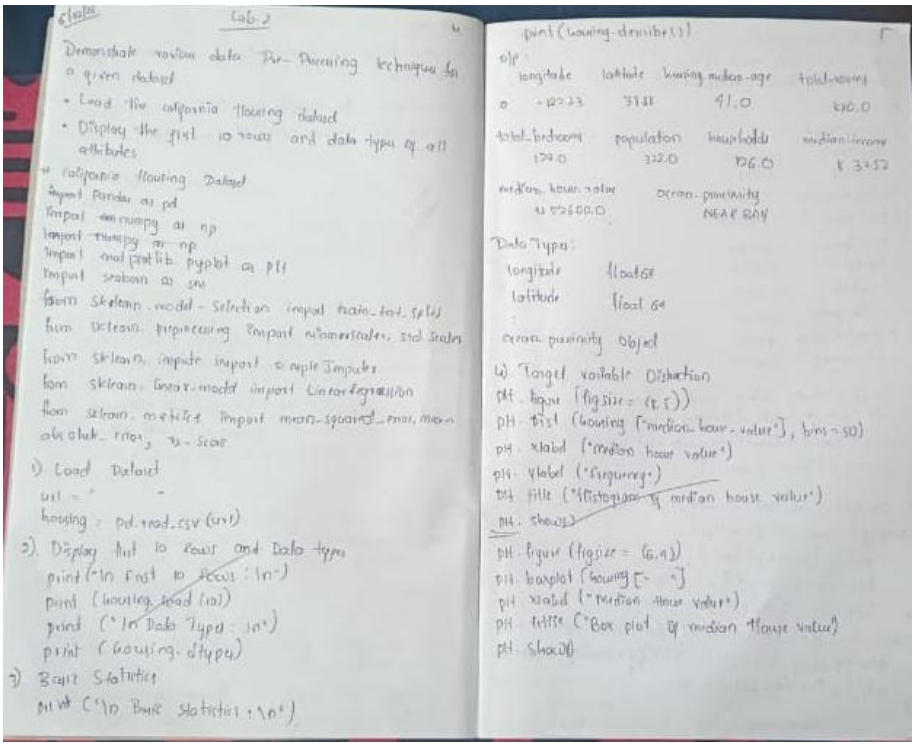
```
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

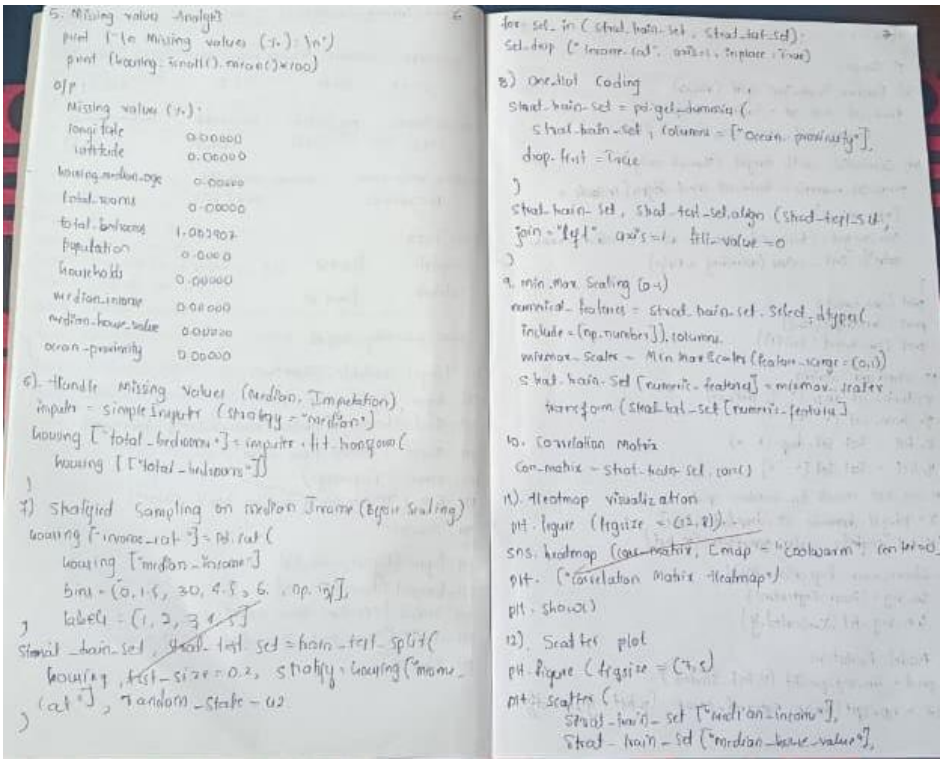
Program 2

Demonstrate various data pre-processing techniques for a given dataset

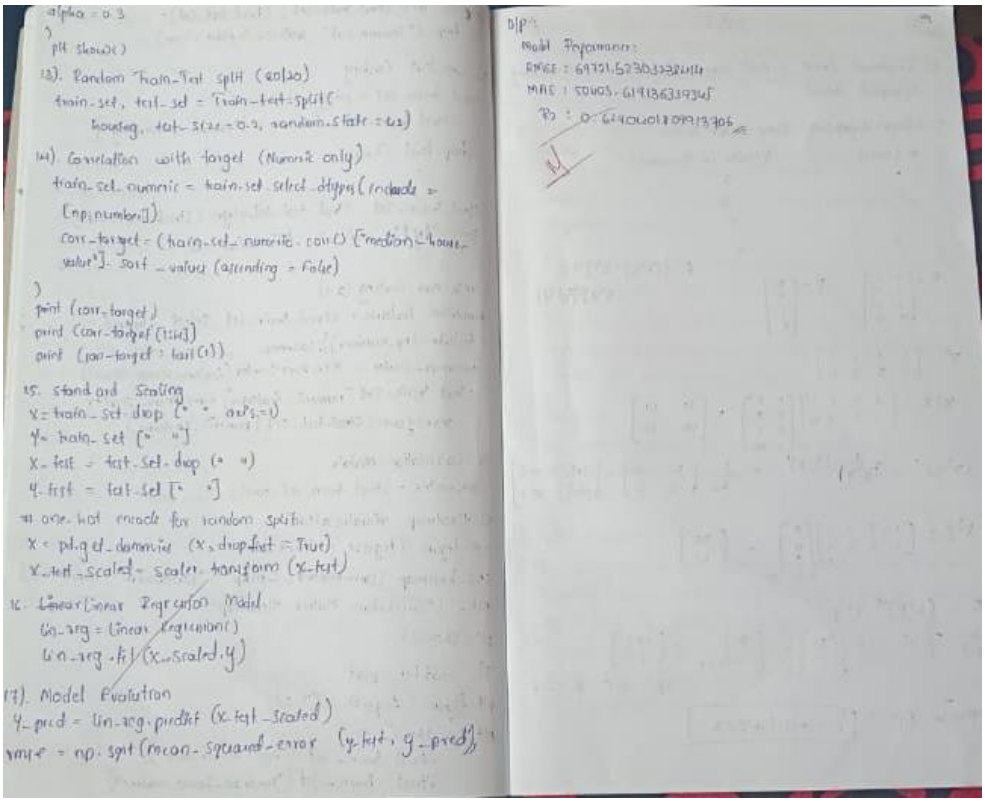
Screenshot



```
Lab-2
Demonstrate various data Pre-Processing techniques for a given dataset
- Load the california Housing dataset
- Display the first 10 rows and data types of all attributes
# California Housing Dataset
import Pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score
1) Load Dataset
url = "
housing = pd.read_csv(url)
2). Display first 10 rows and Data types
print("\n First 10 rows : \n")
print(housing.head(10))
print("\n Data Types : \n")
print(housing.dtypes)
3) Basic Statistics
print("\n Basic Statistics : \n")
print(housing.describe())
o/p
longitude   latitude   housing_median_age   total_rooms
0  -122.23     37.88                 41.0        880.0
total_bedrooms   population   households   median_income
     129.0        322.0        126.0          8.3252
median_house_value   ocean_proximity
          452600.0          NEAR BAY
Data Types:
longitude   float64
latitude    float64
ocean_proximity   object
4) Target variable Distribution
plt.figure(figsize=(8,5))
plt.hist(housing["median_house_value"], bins=50)
plt.xlabel("median house value")
plt.ylabel("frequency")
plt.title("Histogram of median house value")
plt.show()
plt.figure(figsize=(6,4))
plt.boxplot(housing[" "])
plt.xlabel("Median House value")
plt.title("Box plot of median House value")
plt.show()
```



```
5. Missing value Analysis
print("\n Missing values (%) : \n")
print(housing.isnull().mean()*100)
o/p
Missing values (%) :
longitude            0.00000
latitude             0.00000
housing_median_age   0.00000
total_rooms          0.00000
total_bedrooms       1.002907
population           0.00000
households           0.00000
median_income        0.00000
median_house_value   0.00000
ocean_proximity      0.00000
6). Handle Missing Values (Median Imputation)
imputer = SimpleImputer(strategy="median")
housing["total_bedrooms"] = imputer.fit_transform(
    housing[["total_bedrooms"]])
)
7) Stratified Sampling on median Income (Before Scaling)
housing["income_cat"] = pd.cut(
    housing["median_income"]
    bins=[0, 1.5, 3.0, 4.5, 6., np.inf],
    labels=[1, 2, 3, 4, 5]
)
strat_train_set, strat_test_set = train_test_split(
    housing, test_size=0.2, stratify=housing["income_cat"],
    random_state=42
)
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)
8) One-Hot Coding
strat_train_set = pd.get_dummies(
    strat_train_set, columns=["ocean_proximity"],
    drop_first=True
)
strat_train_set, strat_test_set.align(strat_test_set,
    join="left", axis=1, fill_value=0
)
9. min-max Scaling (0-1)
numerical_features = strat_train_set.select_dtypes(
    include=[np.numbers], columns.
minmax_scaler = MinMaxScaler(feature_range=(0,1))
strat_train_set[numeric_features] = minmax_scaler.
    transform(strat_train_set[numeric_features])
10. Correlation Matrix
corr_matrix = strat_train_set.corr()
11). Heatmap visualization
plt.figure(figsize=(12,8))
sns.heatmap(corr_matrix, cmap="coolwarm", annot=0
plt.("Correlation Matrix Heatmap")
plt.show()
12). Scatter plot
plt.figure(figsize=(7,5)
plt.scatter(
    strat_train_set["median_income"],
    strat_train_set["median_house_value"],
```



Code:

```
from sklearn.datasets import load_iris
import pandas as pd
iris = load_iris()
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target
df.head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df.shape
```

```
(150, 5)
```

```
df.columns
```

```
Index(['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)',
       'petal width (cm)', 'species'],
      dtype='object')
```

```python
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   sepal length (cm)  150 non-null    float64
 1   sepal width (cm)   150 non-null    float64
 2   petal length (cm)  150 non-null    float64
 3   petal width (cm)   150 non-null    float64
 4   species            150 non-null    int64
dtypes: float64(4), int64(1)
memory usage: 6.0 KB
```

```python
df.describe()
```

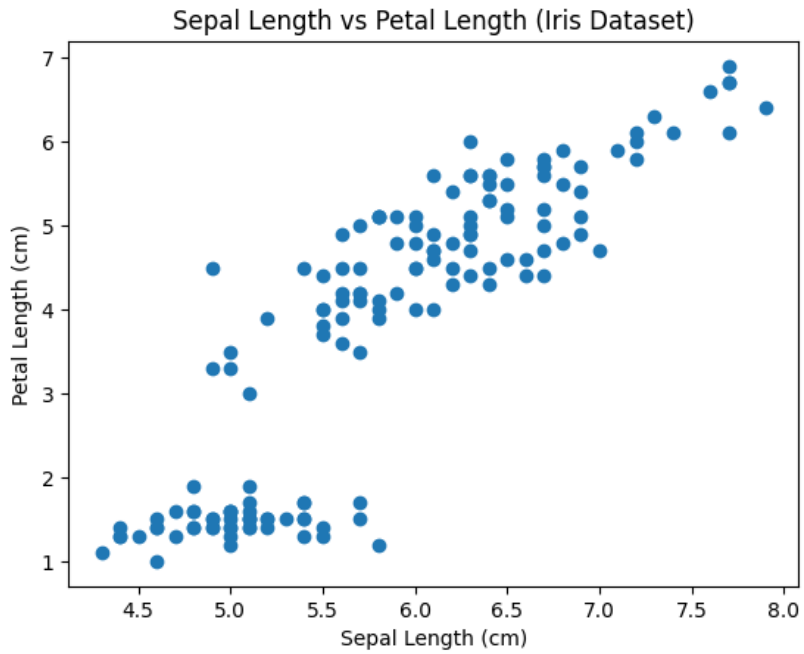
	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
count	150.000000	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333	1.000000
std	0.828066	0.435866	1.765298	0.762238	0.819232
min	4.300000	2.000000	1.000000	0.100000	0.000000
25%	5.100000	2.800000	1.600000	0.300000	0.000000
50%	5.800000	3.000000	4.350000	1.300000	1.000000
75%	6.400000	3.300000	5.100000	1.800000	2.000000
max	7.900000	4.400000	6.900000	2.500000	2.000000

```python
df['species'].value_counts()
```

species	count
0	50
1	50
2	50

dtype: int64

```python
# Import required libraries
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris

# Load the Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['species'] = iris.target

# Scatter plot: Sepal Length vs Petal Length
plt.scatter(df['sepal length (cm)'], df['petal length (cm)'])
plt.xlabel("Sepal Length (cm)")
plt.ylabel("Petal Length (cm)")
plt.title("Sepal Length vs Petal Length (Iris Dataset)")
plt.show()
```




```
df[df['species'] == 0].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
0	5.1	3.5	1.4	0.2	0
1	4.9	3.0	1.4	0.2	0
2	4.7	3.2	1.3	0.2	0
3	4.6	3.1	1.5	0.2	0
4	5.0	3.6	1.4	0.2	0

```
df[df['species'] == 1].head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
50	7.0	3.2	4.7	1.4	1
51	6.4	3.2	4.5	1.5	1
52	6.9	3.1	4.9	1.5	1
53	5.5	2.3	4.0	1.3	1
54	6.5	2.8	4.6	1.5	1

```
df.corr()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
sepal length (cm)	1.000000	-0.117570	0.871754	0.817941	0.782561
sepal width (cm)	-0.117570	1.000000	-0.428440	-0.366126	-0.426658
petal length (cm)	0.871754	-0.428440	1.000000	0.962865	0.949035
petal width (cm)	0.817941	-0.366126	0.962865	1.000000	0.956547
species	0.782561	-0.426658	0.949035	0.956547	1.000000

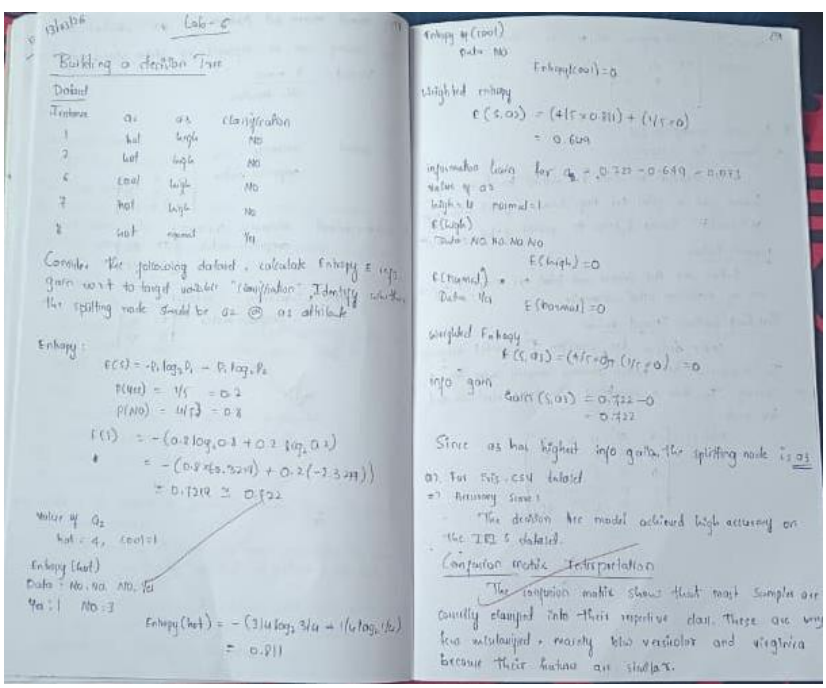
```
df.sort_values(by='petal length (cm)').head()
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	species
22	4.6	3.6	1.0	0.2	0
13	4.3	3.0	1.1	0.1	0
14	5.8	4.0	1.2	0.2	0
35	5.0	3.2	1.2	0.2	0
16	5.4	3.9	1.3	0.4	0

Program 3

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot



overall, the model performs well on this dataset.

$$\text{matrix} = \begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$$

a). for petrol-consumption.csv

=) Regression Tree Interpretation

The regression tree splits the dataset based on imp features such as petrol tax, Avg. Income, paved highway and population Driver License to predict petrol consumption

Important features:-

features like Avg income and Petrol tax play a major role in predicting petrol consumption.

Handling Continuous Target Variable

Unlike decision tree classifier that predict categories The a regression tree predicts continuous numeric values by splitting the data & calculating the avg. value in each leaf node.

$$\text{matrix} = \begin{bmatrix} 6 & 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 & 0 \\ 0 & 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 11 & 0 \\ 0 & 0 & 0 & 0 & 15 \end{bmatrix}$$

Code:

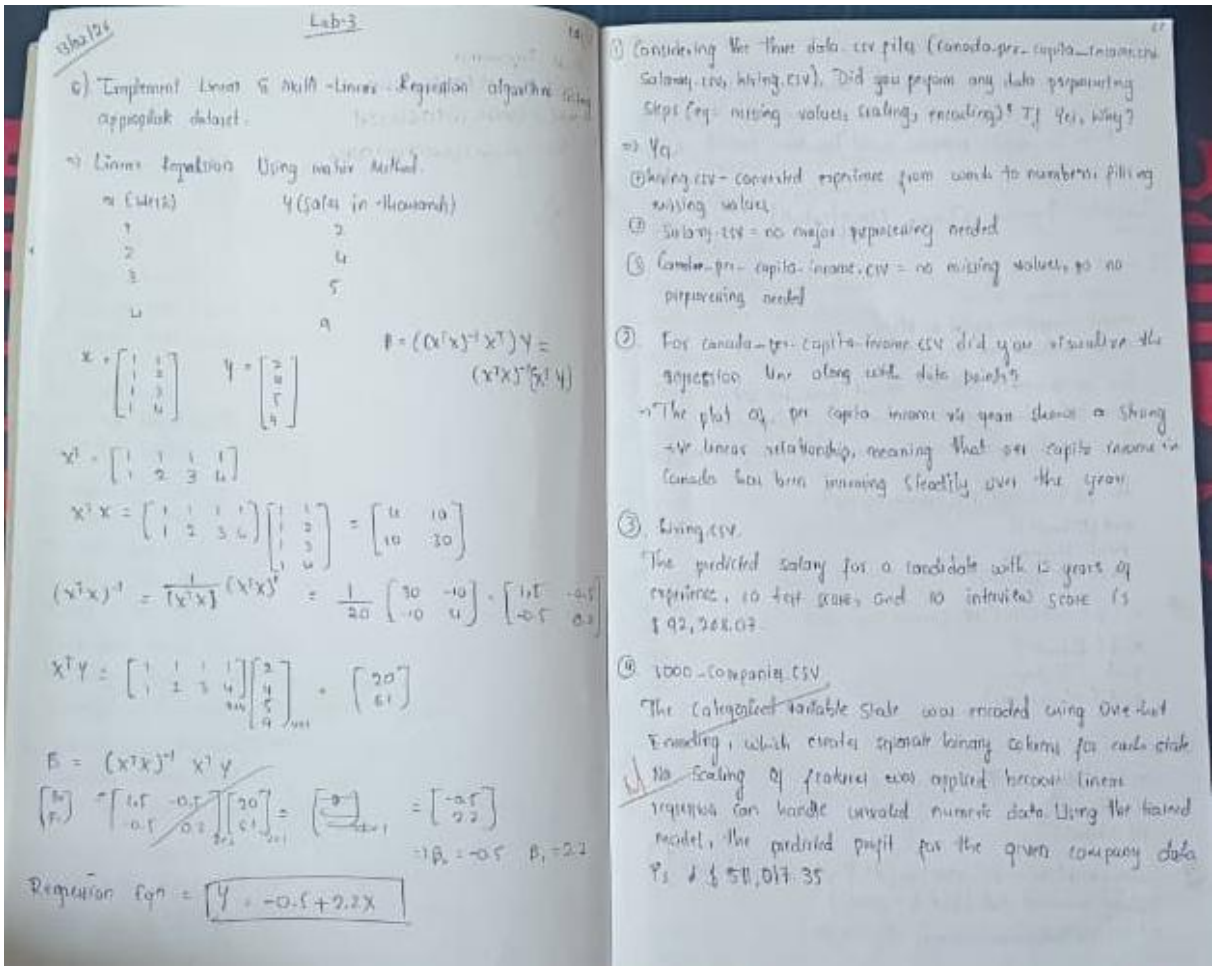
```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = DecisionTreeClassifier(criterion='entropy', random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy:", accuracy_score(y_test, y_pred))

new_sample = [[5.1, 3.5, 1.4, 0.2]]

prediction = model.predict(new_sample)

print("Predicted class:", prediction[0])
print("Predicted species:", iris.target_names[prediction][0])
```

```
Accuracy: 1.0
Predicted class: 0
Predicted species: setosa
```

Program 4

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot



Lab-3

a) Implement Linear & Multi-Linear Regression algorithm using appropriate dataset.

→ Linear Regression Using matrix Method.

x (years)	y (sales in thousands)
1	2
2	4
3	5
4	9

$$\beta = (X^T X)^{-1} X^T Y = (X^T X)^{-1} (X^T Y)$$

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} \quad Y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

$$X^T = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix} = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$$

$$(X^T X)^{-1} = \frac{1}{|X^T X|} (X^T X)^{*} = \frac{1}{20} \begin{bmatrix} 30 & -10 \\ -10 & 4 \end{bmatrix} = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix}$$

$$X^T Y = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 3 & 4 \end{bmatrix}_{2\times4} \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}_{4\times1} = \begin{bmatrix} 20 \\ 61 \end{bmatrix}$$

$$\beta = (X^T X)^{-1} X^T Y$$

$$\begin{bmatrix} \beta_0 \\ \beta_1 \end{bmatrix} = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix}_{2\times2} \begin{bmatrix} 20 \\ 61 \end{bmatrix}_{2\times1} = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix}_{2\times1}$$

$$\Rightarrow \beta_0 = -0.5 \quad \beta_1 = 2.2$$

Regression Eqn = $\boxed{Y = -0.5 + 2.2X}$

① Considering the three data csv files (canada_per_capita_income.csv, Salary.csv, hiring.csv). Did you perform any data preprocessing steps (eg: missing values, scaling, encoding)? If Yes, why?

⇒ Yes.

① hiring.csv - converted experience from words to numbers, filling missing values.

② Salary.csv = no major preprocessing needed.

③ Canada_per_capita_income.csv = no missing values, so no preprocessing needed.

② For canada_per_capita_income.csv did you visualize the regression line along with data points?

→ The plot of per capita income vs year shows a strong +ve linear relationship, meaning that per capita income in Canada has been increasing steadily over the years.

③ hiring.csv.

The predicted salary for a candidate with 12 years of experience, 10 test score, and 10 interview score is $92,268.07

④ 1000_Companies.csv.

The categorical variable state was encoded using One-hot Encoding, which creates separate binary columns for each state.

No scaling of features was applied because linear regression can handle unscaled numeric data. Using the trained model, the predicted profit for the given company data is ₹ $ 51,017.35

Code:

```
X = df.drop('MedHouseVal', axis=1)
y = df['MedHouseVal']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

```
lin_reg = LinearRegression()
lin_reg.fit(X_train_scaled, y_train)

y_pred_lr = lin_reg.predict(X_test_scaled)

rmse_lr = np.sqrt(mean_squared_error(y_test, y_pred_lr))
mae_lr = mean_absolute_error(y_test, y_pred_lr)
r2_lr = r2_score(y_test, y_pred_lr)

rmse_lr, mae_lr, r2_lr
```

```
(np.float64(0.7449540928043491), 0.5330139776645934, 0.5765012224420747)
```

```
dt = DecisionTreeRegressor(random_state=42)
dt.fit(X_train, y_train)

y_pred_dt = dt.predict(X_test)
rmse_dt = np.sqrt(mean_squared_error(y_test, y_pred_dt))

rmse_dt
```

```
np.float64(0.7138377373947808)
```

```
rf = RandomForestRegressor(random_state=42)
rf.fit(X_train, y_train)

y_pred_rf = rf.predict(X_test)
rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

rmse_rf
```

```
np.float64(0.5128644706054181)
```

```
param_grid = {
    'n_estimators': [50, 100],
    'max_depth': [None, 10, 20],
    'min_samples_split': [2, 5]
}

grid_search = GridSearchCV(
    rf,
    param_grid,
    cv=3,
    scoring='neg_mean_squared_error'
)
```

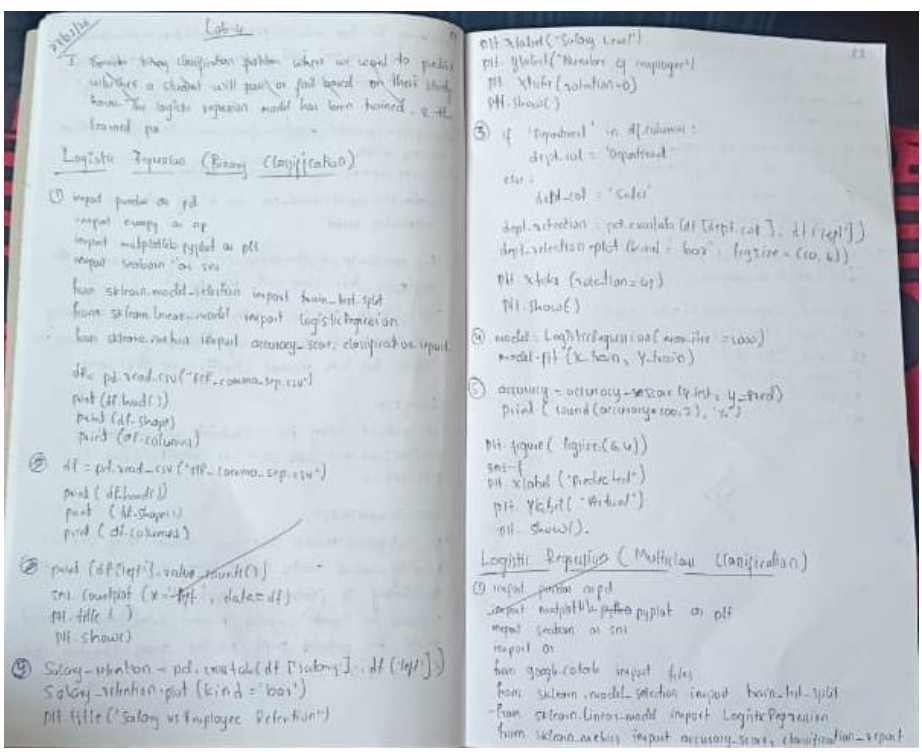
```python
results = pd.DataFrame({
    'Model': ['Linear Regression',
              'Decision Tree',
              'Random Forest',
              'Tuned Random Forest'],
    'RMSE': [rmse_lr, rmse_dt, rmse_rf, rmse_best]
})

results
```

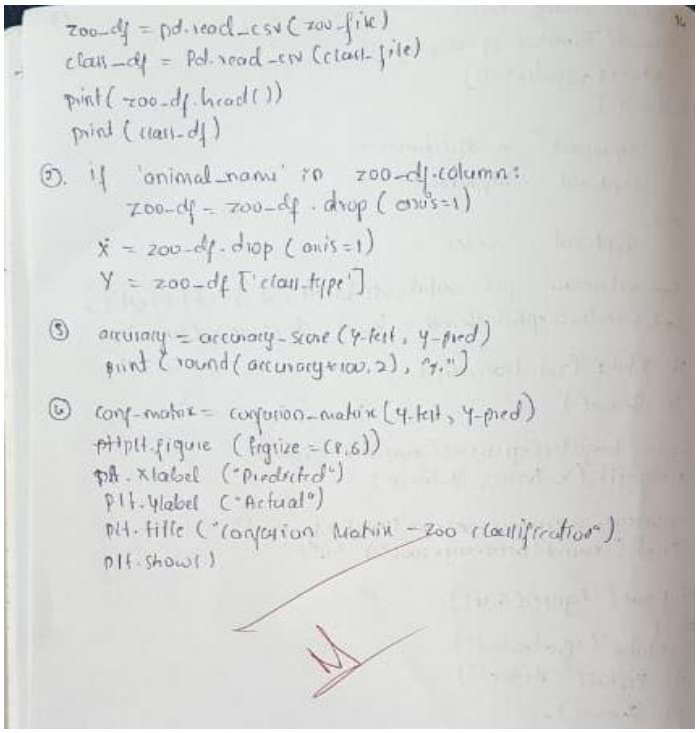
	Model	RMSE
0	Linear Regression	0.744954
1	Decision Tree	0.713838
2	Random Forest	0.512864
3	Tuned Random Forest	0.513263

Program 5

Build Logistic Regression Model for a given dataset

Screenshot





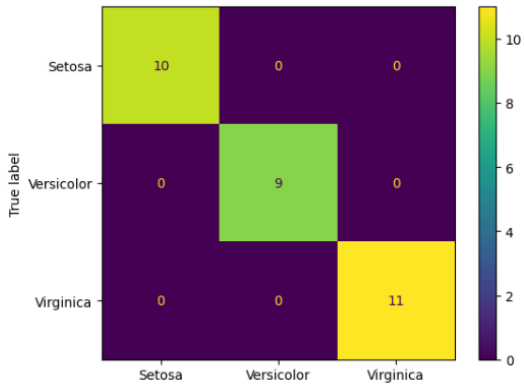
```
zoo_df = pd.read_csv(zoo_file)
class_df = Pd.read_csv(class_file)
print(zoo_df.head())
print(class_df)
5. if 'animal_name' in zoo_df.columns:
    zoo_df = zoo_df.drop(axis=1)
   X = zoo_df.drop(axis=1)
   Y = zoo_df['class_type']
5  accuracy = accuracy_score(y_test, y_pred)
   print(round(accuracy*100,2), "%")
6  conf_matrix = confusion_matrix(y_test, y_pred)
   plt.figure(figsize=(8,6))
   plt.xlabel("Predicted")
   plt.ylabel("Actual")
   plt.title("Confusion Matrix - Zoo classification")
   plt.show()
```

Code:

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn import metrics
import matplotlib.pyplot as plt

# Load correct file
iris = pd.read_csv("iris (1).csv")

print(iris.head())

# Features and target
X = iris.drop('species', axis='columns')
y = iris['species']

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = LogisticRegression(multi_class='multinomial', solver='lbfgs')
model.fit(X_train, y_train)

# Prediction
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```python
# Confusion Matrix
cm = metrics.confusion_matrix(y_test, y_pred)
disp = metrics.ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Setosa", "Versicolor", "Virginica"]
)
disp.plot()
plt.show()
```

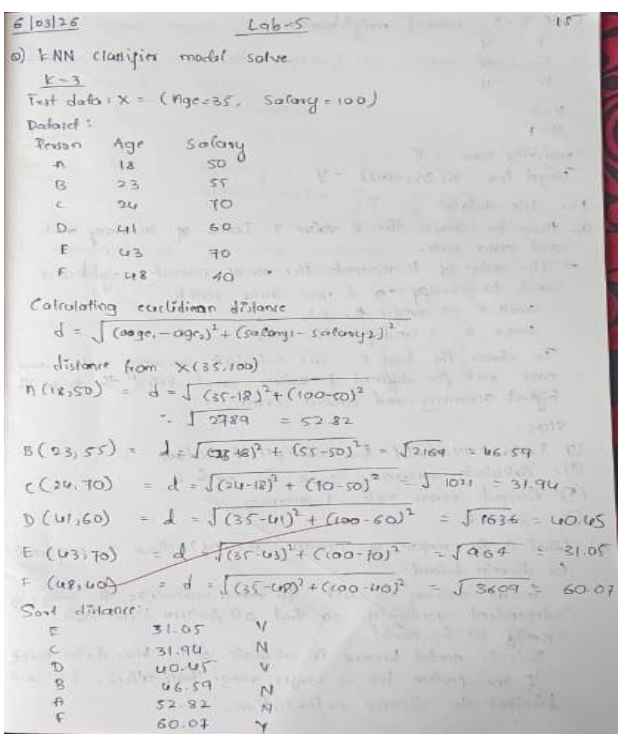
```
   sepal_length  sepal_width  petal_length  petal_width      species
0           5.1          3.5           1.4          0.2  Iris-setosa
1           4.9          3.0           1.4          0.2  Iris-setosa
2           4.7          3.2           1.3          0.2  Iris-setosa
3           4.6          3.1           1.5          0.2  Iris-setosa
4           5.0          3.6           1.4          0.2  Iris-setosa
Accuracy: 1.0
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:1247: FutureWarning: 'multi_class' was deprecated i
  warnings.warn(
```



Program 6

Build KNN Classification model for a given dataset

Screenshot



6/03/26 Lab-5

6) KNN classifier model solve

K=3

Test data: X = (Age=35, Salary=100)

Dataset:

Person	Age	Salary
A	18	50
B	23	55
C	24	70
D	41	60
E	43	70
F	48	40

Calculating euclidean distance

$d = \sqrt{(age_1 - age_2)^2 + (salary_1 - salary_2)^2}$

distance from X(35,100)

A(18,50) = $d = \sqrt{(35-18)^2 + (100-50)^2} = \sqrt{2789} = 52.82$

B(23,55) = $d = \sqrt{(35-18)^2 + (55-50)^2} = \sqrt{2164} = 46.59$

C(24,70) = $d = \sqrt{(24-18)^2 + (70-50)^2} = \sqrt{1021} = 31.94$

D(41,60) = $d = \sqrt{(35-41)^2 + (100-60)^2} = \sqrt{1636} = 40.45$

E(43,70) = $d = \sqrt{(35-43)^2 + (100-70)^2} = \sqrt{964} = 31.05$

F(48,40) = $d = \sqrt{(35-48)^2 + (100-40)^2} = \sqrt{3609} = 60.07$

Sort distance:

E	31.05	Y
C	31.94	N
D	40.45	Y
B	46.59	N
A	52.82	N
F	60.07	Y

Select k=3 nearest neighbours

E	Y
C	N
D	Y

Y = 2
N = 1

majority class = Y
Target for x(35, 100) = Y

For this dataset

Q). How to choose the k value? Demo of accuracy rate and error rate.

=> The value of k represents the no of nearest neighbours used to classify a new data points.

Small k → overfit the data
large k → underfit the data

To choose the best k, we calculate accuracy rate and error rate for different k values, and select the k with highest accuracy and lowest errors.

Steps:

1. Train KNN model for multiple k values
2. Calculate accuracy score for each k
3. Compute errors rate = 1 - accuracy
4. Select k

Q) What is the purpose of feature scaling? How to perform it for diabetes dataset.

feature scaling is used to ~~norm~~ normalize the range of independent variables so that all features contribute equally to the model.

This is needed because it calculate dist. b/w data points if one feature has a larger range than other. it will dominate the distance calculation.

It ensures all features are on similar scale.

feature scaling can be performed by Stdar Standardization

$$X_{scaled} = \frac{X - mean}{\text{Std. deviation}}$$

Results:

for iris dataset = accuracy : 1.0

confusion matrix $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

for diabetes dataset = accuracy : 0.6948051

confusion matrix : $\begin{bmatrix} 79 & 20 \\ 27 & 28 \end{bmatrix}$

for heart dataset : accuracy : 0.9180327

confusion matrix : $\begin{bmatrix} 27 & 2 \\ 3 & 29 \end{bmatrix}$

Code:

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import mean_squared_error, r2_score

from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import DecisionTreeRegressor
```

```
from sklearn.datasets import load_iris

data = load_iris()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
model = DecisionTreeClassifier(criterion='entropy')

model.fit(X_train, y_train)

y_pred = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print(classification_report(y_test, y_pred))
```

```
Accuracy: 1.0
              precision    recall  f1-score   support

           0       1.00      1.00      1.00        19
           1       1.00      1.00      1.00        13
           2       1.00      1.00      1.00        13

    accuracy                           1.00        45
   macro avg       1.00      1.00      1.00        45
weighted avg       1.00      1.00      1.00        45
```

```
from sklearn.datasets import load_wine

data = load_wine()

X = data.data
y = data.target
```

```
from sklearn.datasets import load_diabetes

data = load_diabetes()

X = data.data
y = data.target
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

```
regressor = DecisionTreeRegressor()

regressor.fit(X_train, y_train)

y_pred = regressor.predict(X_test)
```

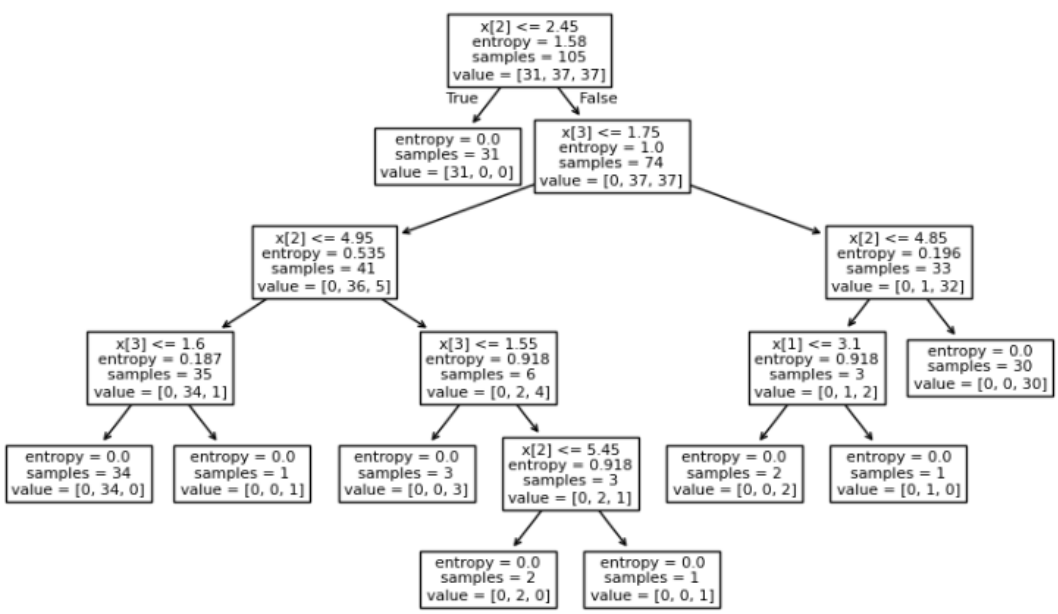
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

print("Mean Squared Error:", mse)
print("R2 Score:", r2)
```

```
Mean Squared Error: 5358.458646616541
R2 Score: 0.0073817790735694055
```

```
from sklearn.datasets import fetch_california_housing

data = fetch_california_housing()

X = data.data
y = data.target
```

```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))
plot_tree(model)
plt.show()
```

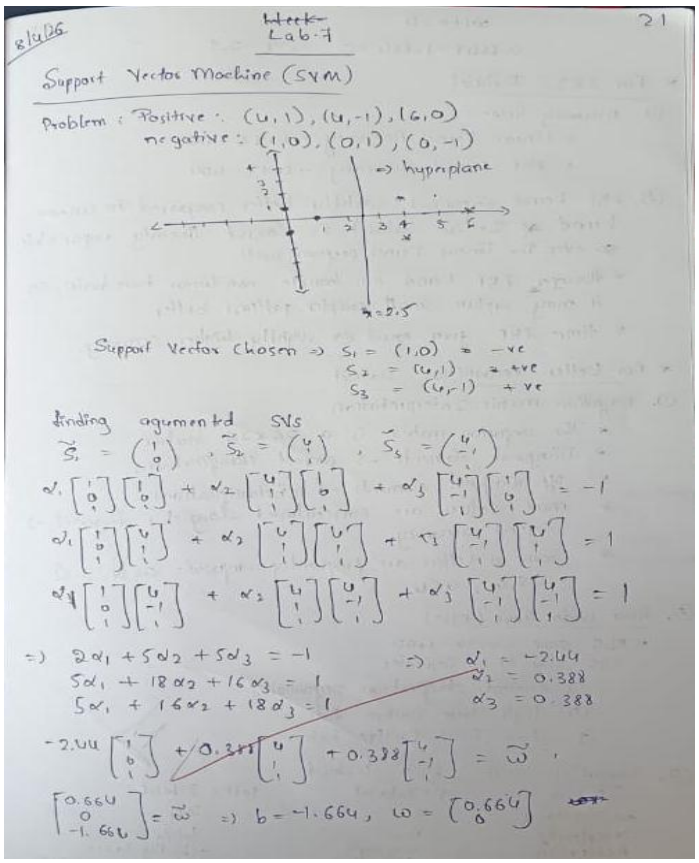
...



Program 7

Build Support vector machine model for a given dataset

Screenshot



8/4/26 ~~Week~~ Lab-7 21

Support Vector Machine (SVM)

Problem : Positive : (4,1), (4,-1), (6,0)
negative : (1,0), (0,1), (0,-1)

⇒ hyperplane

$x = 2.5$

Support Vector Chosen ⇒ $S_1 = (1,0)$ = -ve
$S_2 = (4,1)$ = +ve
$S_3 = (4,-1)$ + ve

finding augmented SVs

$\tilde{S}_1 = \begin{pmatrix}1\\0\\1\end{pmatrix}$, $\tilde{S}_2 = \begin{pmatrix}4\\1\\1\end{pmatrix}$, $\tilde{S}_3 = \begin{pmatrix}4\\-1\\1\end{pmatrix}$

$$\alpha_1 \begin{bmatrix}1\\0\\1\end{bmatrix}\begin{bmatrix}1\\0\\1\end{bmatrix} + \alpha_2 \begin{bmatrix}4\\1\\1\end{bmatrix}\begin{bmatrix}1\\0\\1\end{bmatrix} + \alpha_3 \begin{bmatrix}4\\-1\\1\end{bmatrix}\begin{bmatrix}1\\0\\1\end{bmatrix} = -1$$

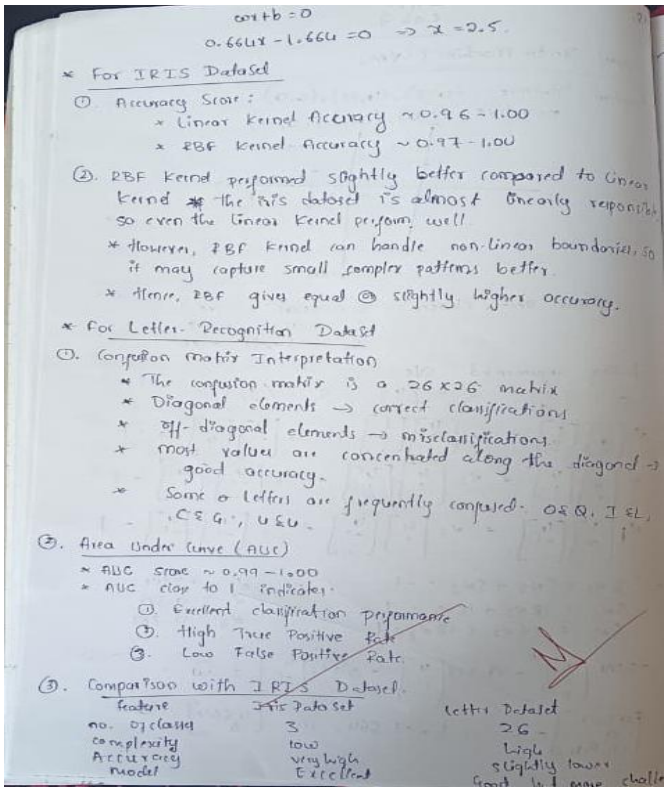
$$\alpha_1 \begin{bmatrix}1\\0\\1\end{bmatrix}\begin{bmatrix}4\\1\\1\end{bmatrix} + \alpha_2 \begin{bmatrix}4\\1\\1\end{bmatrix}\begin{bmatrix}4\\1\\1\end{bmatrix} + \alpha_3 \begin{bmatrix}4\\-1\\1\end{bmatrix}\begin{bmatrix}4\\1\\1\end{bmatrix} = 1$$

$$\alpha_1 \begin{bmatrix}1\\0\\1\end{bmatrix}\begin{bmatrix}4\\-1\\1\end{bmatrix} + \alpha_2 \begin{bmatrix}4\\1\\1\end{bmatrix}\begin{bmatrix}4\\-1\\1\end{bmatrix} + \alpha_3 \begin{bmatrix}4\\-1\\1\end{bmatrix}\begin{bmatrix}4\\-1\\1\end{bmatrix} = 1$$

$$\Rightarrow \begin{aligned} 2\alpha_1 + 5\alpha_2 + 5\alpha_3 &= -1 \\ 5\alpha_1 + 18\alpha_2 + 16\alpha_3 &= 1 \\ 5\alpha_1 + 16\alpha_2 + 18\alpha_3 &= 1 \end{aligned} \quad \Rightarrow \quad \begin{aligned} \alpha_1 &= -2.44 \\ \alpha_2 &= 0.388 \\ \alpha_3 &= 0.388 \end{aligned}$$

$$-2.44 \begin{bmatrix}1\\0\\1\end{bmatrix} + 0.388 \begin{bmatrix}4\\1\\1\end{bmatrix} + 0.388 \begin{bmatrix}4\\-1\\1\end{bmatrix} = \tilde{\omega}$$

$$\begin{bmatrix}0.664\\0\\-1.664\end{bmatrix} = \tilde{\omega} \Rightarrow b = -1.664, \quad \omega = \begin{bmatrix}0.664\\0\end{bmatrix}$$



$wx + b = 0$

$0.664x - 1.664 = 0 \rightarrow x = 2.5$

* For IRIS Dataset

① Accuracy Score :
- * Linear Kernel Accuracy ~0.96 - 1.00
- * RBF Kernel Accuracy ~0.97 - 1.00

②. RBF Kernel preformed slightly better compared to Linear Kernel * the iris dataset is almost linearly separable so even the linear kernel preform well
- * However, RBF kernel can handle non-linear boundaries, so it may capture small complex patterns better.
- * Hence, RBF gives equal @ slightly higher accuracy.

* For Letter Recognition Dataset

①. Confusion matrix Interpretation
- * The confusion matrix is a 26 x 26 matrix
- * Diagonal elements → correct classifications
- * off-diagonal elements → misclassifications
- * most values are concentrated along the diagonal → good accuracy.
- * Some of letters are frequently confused. O & Q, I & L, C & G, U & V.

②. Area Under Curve (AUC)
- * ABC score ~ 0.99 - 1.00
- * AUC close to 1 indicates:
  - ①. Excellent classification preformance
  - ②. High True Positive Rate
  - ③. Low False Positive Rate.

③. Comparison with IRIS Dataset.

feature	Iris Data set	letter Dataset
no. of classes	3	26
complexity	low	high
Accuracy	very high	slightly lower
model	Excellent	Good but more challe

Code:

```python
# Import libraries
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.preprocessing import StandardScaler

# Load dataset
data = datasets.load_iris()
X = data.data
y = data.target

# Feature scaling (important for SVM)
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Split dataset (fixed for same results every time)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create SVM model
model = SVC(kernel='linear')

# Train model
model.fit(X_train, y_train)

# Predict
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)
```

```python
# Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
print("\nConfusion Matrix:\n", cm)

# Classification Report
cr = classification_report(y_test, y_pred)
print("\nClassification Report:\n", cr)
```

```
Accuracy: 0.9666666666666667

Confusion Matrix:
 [[10  0  0]
 [ 0  8  1]
 [ 0  0 11]]

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00        10
           1       1.00      0.89      0.94         9
           2       0.92      1.00      0.96        11

    accuracy                           0.97        30
   macro avg       0.97      0.96      0.97        30
weighted avg       0.97      0.97      0.97        30
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot

8/10/26

23

Lab-8

Random Forest

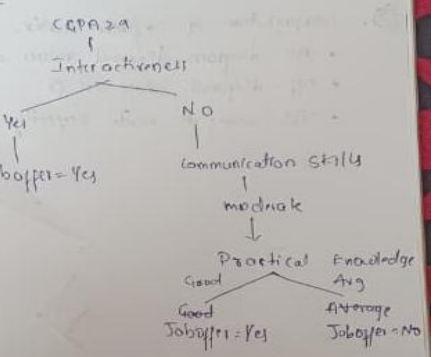
Problem : 1. Sample S1 - Root Node : CGPA

From the table :

- All entries have CGPA ≥ 9
- So, CGPA does not split data further

So, best attribute : Intreactiveness

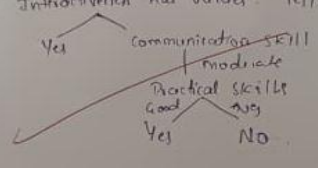
Decision Tree



2). Sample S2 - Root Node : Intreactiveness

from table :

① Intreactiveness has values : Yes/No



24

× For Iris Dataset

① Best Accuracy Cor Score :

- The best accuracy observed was : ≈ 0.95 to 1.00
- This was achieved using around 50 - 100 trees.

② Confusion matrix :

Eg (when accuracy = 1.0) :

$$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$$

③. Interpretation of confusion matrix.

- All diagonal values are high → correct classification
- Off - diagonal values are 0
- This means the model perfectly classified all samples.

Code:

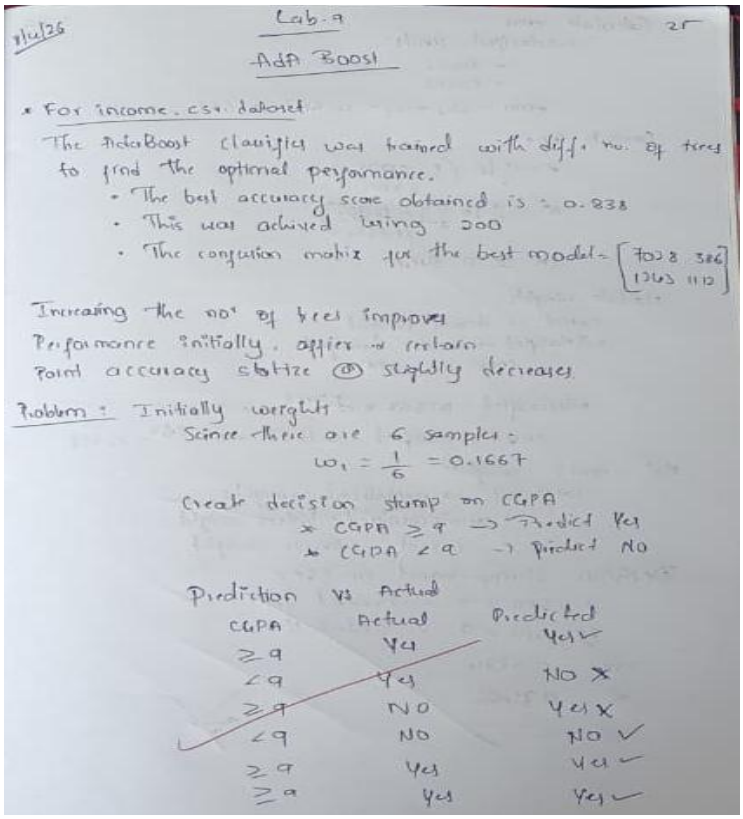
```python
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()

X = iris.data
y = iris.target

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

model = RandomForestClassifier(n_estimators=100, random_state=42)

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
```

```
Accuracy: 1.0
Confusion Matrix:
 [[10  0  0]
 [ 0  9  0]
 [ 0  0 11]]
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot



Lab-9

AdA Boost

- For income.csv dataset

The AdaBoost classifier was trained with diff. no. of trees to find the optimal performance.

- The best accuracy score obtained is = 0.838
- This was achieved using = 200
- The confusion matrix for the best model = $\begin{bmatrix} 7028 & 386 \\ 1363 & 1110 \end{bmatrix}$

Increasing the no. of trees improves Performance initially, after a certain point accuracy stabilize or slightly decreases.

Problem : Initially weights

Since there are 6 samples :

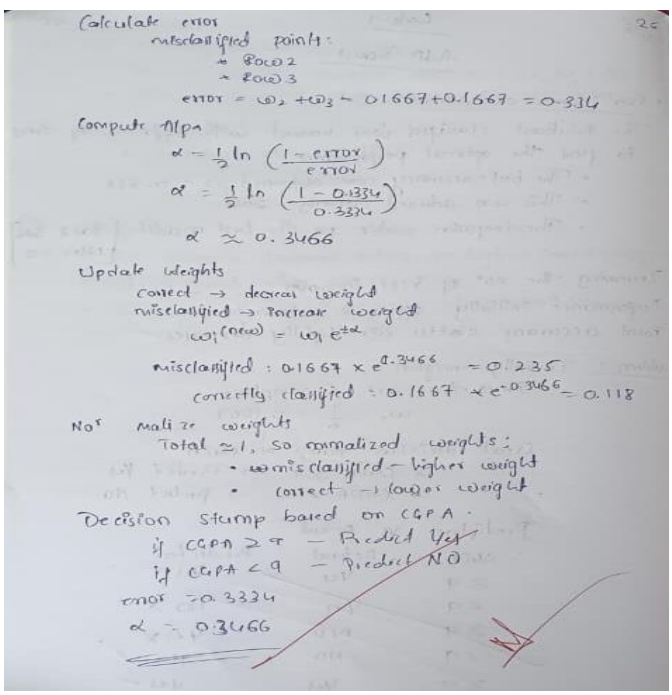
$w_1 = \frac{1}{6} = 0.1667$

Create decision stump on CGPA

- CGPA ≥ 9 → Predict Yes
- CGPA < 9 → Predict NO

Prediction vs Actual

CGPA	Actual	Predicted
≥ 9	Yes	Yes ✓
< 9	Yes	NO ✗
≥ 9	NO	Yes ✗
< 9	NO	NO ✓
≥ 9	Yes	Yes ✓
≥ 9	Yes	Yes ✓



Calculate error

misclassified points :

- Row 2
- Row 3

error $= w_2 + w_3 = 0.1667 + 0.1667 = 0.334$

Compute Alpha

$\alpha = \frac{1}{2} \ln \left(\frac{1 - \text{error}}{\text{error}} \right)$

$\alpha = \frac{1}{2} \ln \left(\frac{1 - 0.1334}{0.3334} \right)$

$\alpha \approx 0.3466$

Update weights

correct → decrease weight

misclassified → increase weight

$w_i^{(new)} = w_i e^{\pm\alpha}$

misclassified : $0.1667 \times e^{0.3466} = 0.235$

correctly classified : $0.1667 \times e^{-0.3466} = 0.118$

No. Normalize weights

Total ≈ 1, so normalized weights :

- misclassified – higher weight
- correct → lower weight

Decision stump based on CGPA.

if CGPA ≥ 9 – Predict Yes

if CGPA < 9 – Predict NO

error = 0.3334

α = 0.3466

Code:

```python
# Import libraries
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, classification_report

# Load Wine dataset
data = load_wine()
X = data.data
y = data.target

# Split dataset into train and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create AdaBoost model
model = AdaBoostClassifier(
    n_estimators=50,      # number of weak learners
    learning_rate=1.0,    # contribution of each classifier
    random_state=42
)

# Train model
model.fit(X_train, y_train)

# Make predictions
y_pred = model.predict(X_test)

# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

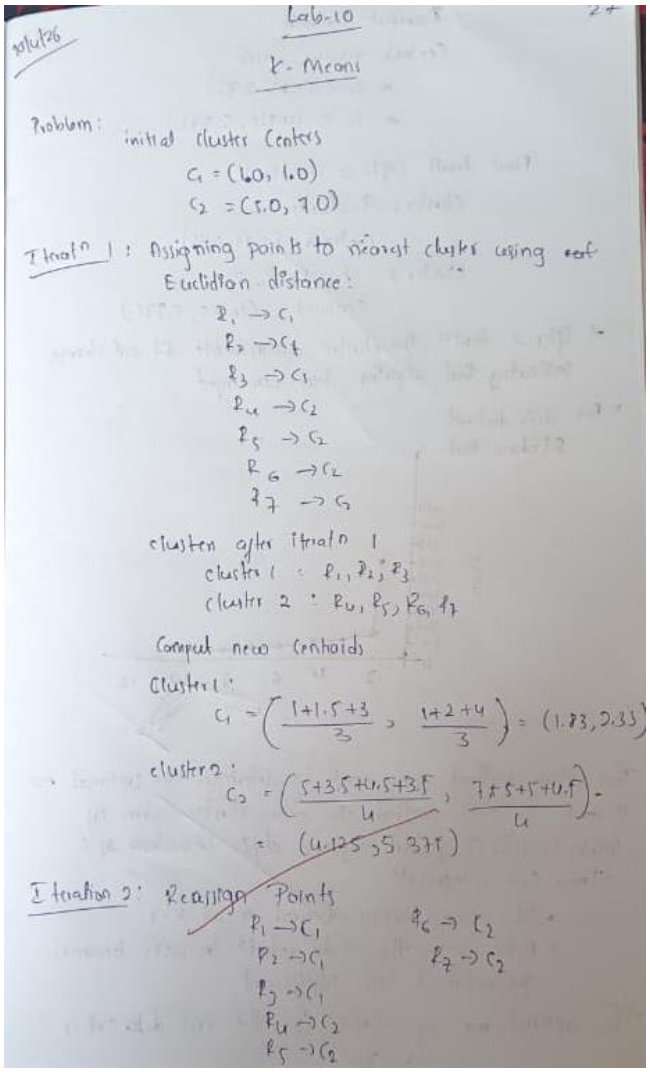
```python
# Evaluate model
print("Accuracy:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

```
Accuracy: 0.9444444444444444

Classification Report:
               precision    recall  f1-score   support

           0       0.93      1.00      0.97        14
           1       0.93      0.93      0.93        14
           2       1.00      0.88      0.93         8

    accuracy                           0.94        36
   macro avg       0.95      0.93      0.94        36
weighted avg       0.95      0.94      0.94        36
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot



Lab-10

K-Means

Problem: initial cluster Centers

$C_1 = (1.0, 1.0)$

$C_2 = (5.0, 7.0)$

Itrat^n 1: Assigning points to nearest cluster using Euclidian distance:

$P_1 \rightarrow C_1$

$P_2 \rightarrow C_1$

$P_3 \rightarrow C_1$

$P_4 \rightarrow C_2$

$P_5 \rightarrow C_2$

$P_6 \rightarrow C_2$

$P_7 \rightarrow C_2$

cluster after itrat^n 1

cluster 1 : P_1, P_2, P_3

cluster 2 : P_4, P_5, P_6, P_7

Compute new centroids

Cluster 1:

$$C_1 = \left(\frac{1+1.5+3}{3}, \frac{1+2+4}{3}\right) = (1.83, 2.33)$$

cluster 2:

$$C_2 = \left(\frac{5+3.5+4.5+3.5}{4}, \frac{7+5+5+4.5}{4}\right) = (4.125, 5.375)$$

Itration 2: Reassign Points

$P_1 \rightarrow C_1$ $P_6 \rightarrow C_2$

$P_2 \rightarrow C_1$ $P_7 \rightarrow C_2$

$P_3 \rightarrow C_1$

$P_4 \rightarrow C_2$

$P_5 \rightarrow C_2$

Recompute Centroids

Centroids remain Same

* $C_1 = (1.83, 2.33)$
* $C_2 = (4.125, 5.375)$

Final Result (after 2 iterations):

Cluster 1: P_1, P_2, P_3

Centroid = (1.83, 2.33)

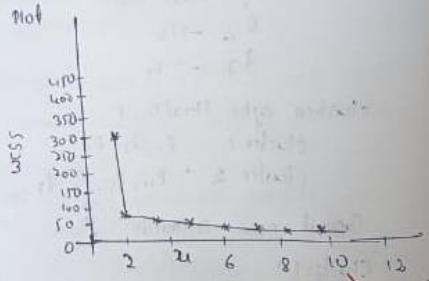
cluster 2: P_4, P_5, P_6, P_7

Centroid = (4.125, 5.375)

⇒ After 2 iterations, the cluster assignments did not change, indicating that algorithm has converged.

* For Iris dataset

Elbow Plot



The elbow method was used to determin the optimal no of clusters k. In this method, the within cluster sum of square (WCSS) is plotted against diff values of k.

Elbow Plot interpretation

* The elbow points observed at k = 3
* After k = 3, the rectal reduction in WCSS becomes gradated & less significant

The optimal no. of clusters for the iris data set is k = 3

Code:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
```

```
data = pd.read_csv("/content/data.csv")
print(data.head())
```

```
X = data.iloc[:, :].values   # or choose specific columns
```

```
k = 3   # you can change this
```

```
kmeans = KMeans(n_clusters=k, random_state=0)
kmeans.fit(X)
```

```
labels = kmeans.labels_
print(labels)
```

```
centroids = kmeans.cluster_centers_
print(centroids)
```

```
plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.scatter(centroids[:, 0], centroids[:, 1], s=200, c='red')
plt.title("K-Means Clustering")
plt.show()
```

Program 11

Implement Dimensionality reduction using Principle Component Analysis (PCA) method.

Screenshot

10/4/26

Lab-11

PCA

Problem:

Given: $x_1 = [4 \quad 8 \quad 13 \quad 7]$

$x_2 = [11 \quad 4 \quad 5 \quad 14]$

from x: $x = \begin{bmatrix} 4 & 8 & 13 & 7 \\ 11 & 4 & 5 & 14 \end{bmatrix}$

compute mean of each feature:

$$\bar{x}_1 = \frac{4+8+13+7}{4} = 8$$

$$\bar{x}_2 = \frac{11+4+5+14}{4} = 8.5$$

Centre the data:

Subtract mean from each value

$x_1' = [-4 \quad 0 \quad 5 \quad -1]$

$x_2' = [2.5 \quad -4.5 \quad -3.5 \quad 5.5]$

use first eigenvectors:

$$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

Compute first PCA:

$PC_1 = 0.5574 \times x_1' + (0.8303)\, x_2'$

$eg_1 = PC_1 = (0.5574 \times -4) + (-0.8303 \times 2.5)$

$= -4.30536$

$eg_2 = PC_1 = (0.5574 \times 6) + (-0.8303 \times -4.5)$

$= 3.73635$

$eg_3 = PC_1 = (0.5574 \times 5) + (-0.8303 \times -3.5)$

$= 5.69305$

$eg_4 = PC_1 = (0.5574 \times -1) + (-0.8303 \times 5.5)$

$= -5.12405$

$PC_1 = [-4.305, 3.736, 5.6930, -5.124]$

* For breast.csv dataset

① Accuracy Before applying PCA:

* SVM: 0.875
* Logistic Regression: 0.85326
* Random forest: 0.85869

② Accuracy After applying PCA:

* SVM: 0.875
* Logistic Regression: 0.85326
* Random Forest: 0.86413.

Code:

PCA

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler
```

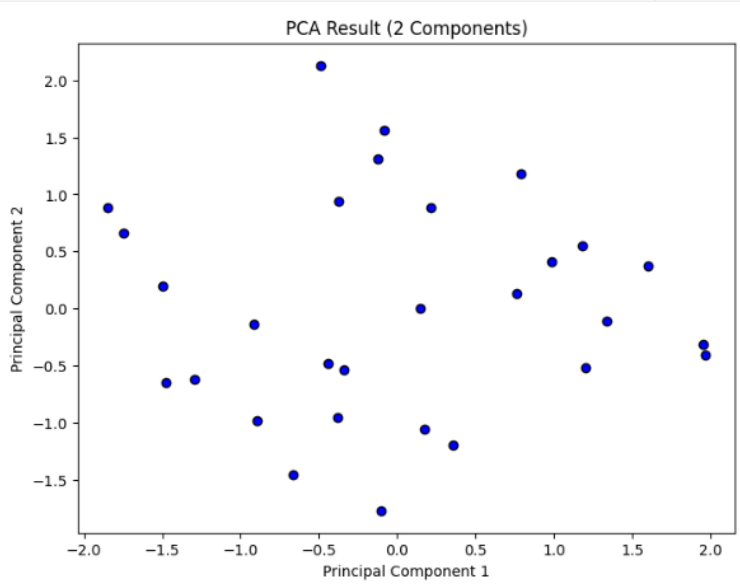
```
# Load dataset
data = pd.read_csv("data.csv")
print(data.head())
```

```
    x   y
0  30  49
1  31  27
2  31  46
3   1  65
4  81  76
```

```
# Standardize the data
scaler = StandardScaler()
scaled_data = scaler.fit_transform(data)
```

```
# Apply PCA
pca = PCA(n_components=2)  # Reduce to 2 components
pca_result = pca.fit_transform(scaled_data)
```

```
# Explained variance ratio
print(f"Explained Variance Ratio: {pca.explained_variance_ratio_}")
print(f"Total Variance Explained by 2 Components: {np.sum(pca.explained_variance_ratio_)}")
```

```
Explained Variance Ratio: [0.56506606 0.43493394]
Total Variance Explained by 2 Components: 1.0
```

```
# Scatter plot of the 2D PCA result
plt.figure(figsize=(8, 6))
plt.scatter(pca_result[:, 0], pca_result[:, 1], c='blue', edgecolors='black')
plt.title('PCA Result (2 Components)')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.show()
```